GUIDE PRATIQUE · ÉDITION 2.0 · 31 OCTOBRE 2026

L'IA locale sur Mac

Assistant conversationnel, transcription et recherche documentaire sur votre Mac — sans cloud, sans abonnement, sans fuite de données.

Votre IA privée en 30 minutes | 6 cas métiers pas à pas | Vitesses mesurées et sourcées | Kit : scripts, prompts, RGPD

Ollama 0.40.13 · LM Studio · MLX · Whisper · RAG · RGPD

Yann Abadie

Sommaire détaillé

Démarrage express : votre IA privée en 30 minutes

Étape 1 — Vérifier le Mac (3 min)
Étape 2 — Installer Ollama (5 min)
Étape 3 — Télécharger le bon modèle (≈ 12 min, selon la connexion)
Étape 4 — Première vraie question (3 min)
Étape 5 — Le test « mode avion » (3 min)
Étape 6 — Votre assistant français (4 min)

Avant de commencer

À qui s'adresse ce guide
Ce que vous saurez faire à la fin
Ce que ce guide n'est pas
Conventions
Le kit fourni

1. Pourquoi faire tourner l'IA en local

Vos données restent chez vous
Un coût marginal nul
Disponibilité et maîtrise
Les limites honnêtes
Pourquoi le Mac est particulièrement adapté

2. Choisir son Mac et sa mémoire

La règle de calcul
Les modèles « MoE » changent la donne
Que faire avec quelle mémoire (octobre 2026)
Le disque
Vérifier votre Mac
Et si mon Mac a une puce Intel ?

2 bis. Les chiffres : vitesse et mémoire mesurées

La règle de mémoire, en une ligne
La vitesse dépend surtout de la bande passante mémoire
Comment lire ce tableau
Mesurer votre propre Mac

3. Installer les outils

3.1 Prérequis : Homebrew (optionnel mais pratique)
3.2 Ollama
3.3 LM Studio
3.4 llama.cpp
3.5 MLX et mlx-lm
3.6 Tableau de décision

4. Choisir ses modèles

Lire le nom d'un modèle
La quantisation, simplement
Le contexte
Sélection d'octobre 2026
Tester un modèle en 10 minutes
Les « modèles qui réfléchissent »

5. Transcrire ses réunions en local

5.1 Les deux options
5.2 Préparer l'audio
5.3 whisper.cpp
5.4 MLX Whisper
5.5 De la transcription au compte rendu
5.6 Bonnes pratiques
5.7 Dictée au quotidien

6. Interroger ses documents : le RAG local

6.1 Le principe en 4 étapes
6.2 Option clé en main : Open WebUI
6.3 Autres options graphiques
6.4 Option « je maîtrise tout » : le script du kit
6.5 Préparer ses documents
6.6 Limites à connaître

7. Automatiser : Raccourcis, scripts et API

7.1 Un service « toujours prêt »
7.2 Raccourcis macOS
7.3 Traiter un dossier entier
7.4 Python en 10 lignes
7.5 Sorties structurées
7.6 Outils de développement

8. RGPD, secret professionnel et bonnes pratiques

8.1 Ce que le local change… et ne change pas
8.2 Professions soumises au secret
8.3 Le règlement européen sur l'IA (AI Act)
8.4 Sécuriser le Mac
8.5 Documenter (registre et charte)
8.6 Le piège des « hybrides »

9. Six cas concrets, pas à pas

9.1 Avocat : chronologie des faits à partir de pièces
9.2 Expert-comptable : extraire une facture vers un tableur
9.3 Notaire : comparer deux versions d'un acte
9.4 Kinésithérapeute ou médecin : dictée → compte rendu structuré
9.5 PME : compte rendu de réunion avec actions
9.6 Toute profession : répondre à un courriel délicat
9.7 Un serveur pour l'équipe
9.8 Mesurer le gain avant de généraliser

10. Dépannage et performances

10.1 Problèmes fréquents
10.2 Diagnostiquer
10.3 Gagner en vitesse
10.4 Mettre à jour

10 bis. Arbre de dépannage et preuve de confidentialité

Partir du symptôme
Prouver que rien ne sort de votre Mac

11. Checklists

Installation (1 heure)
Transcription
RAG
Conformité
Avant chaque usage sensible

Annexe A — Glossaire

Annexe B — Contenu du kit

Le script ia.sh
Le cœur du RAG (extrait de rag_local.py)
Prompt « compte rendu »
Programme de prise en main sur 5 jours

Annexe C — Sources

Édition, mises à jour et contact

Cette édition
Mises à jour
Une erreur, une question ?

Démarrage express : votre IA privée en 30 minutes

Ce parcours vous amène, sans rien comprendre de technique, à une IA qui répond **sur votre Mac, Wi-Fi coupé**. Chaque étape se termine par un **point de contrôle** : si vous ne voyez pas ce qui est décrit, allez directement à la ligne correspondante du chapitre 10.

Minute	Étape	Point de contrôle
0–3	Vérifier le Mac	Vous connaissez votre puce et votre mémoire
3–8	Installer Ollama	L'icône du lama est dans la barre des menus
8–20	Télécharger **un** modèle adapté	`ollama list` affiche le modèle
20–23	Première vraie question	Une réponse en français s'affiche
23–26	Test « mode avion »	La réponse arrive Wi-Fi coupé
26–30	Créer votre assistant français	`ollama run assistant-fr` répond en français, sans formule d'introduction

Étape 1 — Vérifier le Mac (3 min)

Menu › **À propos de ce Mac**. Notez la **puce** (M1, M2… M5) et la **mémoire** (8, 16, 24, 32 Go…). Une puce Intel ne convient pas à ce guide.

Ou, dans le Terminal (Applications › Utilitaires › Terminal), avec le script du kit :

```
bash ~/ia-locale-kit/scripts/check-mac.sh
```

Étape 2 — Installer Ollama (5 min)

1. Ouvrez https://ollama.com/download, cliquez **Download for macOS** (version 0.40.13 au 31/10/2026).
2. Ouvrez le fichier téléchargé, glissez **Ollama** dans **Applications**, lancez-le et acceptez l'installation de la commande `ollama`.
3. Dans le Terminal : `ollama --version`.

> **Point de contrôle** — une ligne `ollama version is 0.40.x` s'affiche. Sinon : chapitre 10, ligne « command not found ».

Étape 3 — Télécharger le bon modèle (≈ 12 min, selon la connexion)

Ne téléchargez **qu'un seul** modèle pour commencer, choisi selon votre mémoire :

Mémoire du Mac	Commande	Taille téléchargée (ordre de grandeur)
8 Go	`ollama pull gemma4:e4b`	~ 3–4 Go
16 Go	`ollama pull gemma4:12b`	~ 8 Go
24–32 Go	`ollama pull qwen3.6:35b`	~ 20+ Go
48 Go et plus	`ollama pull qwen3.6:35b` puis, au besoin, `mistral-small3.2:24b`	—

Tailles indicatives ; la valeur exacte s'affiche sur la page du modèle (ollama.com/library) et pendant le téléchargement.

Étape 4 — Première vraie question (3 min)

Choisissez une tâche **réelle** de votre journée, pas « bonjour » :

```
ollama run gemma4:12b "Reformule ce message pour un client mécontent, ton
courtois et ferme, 5 lignes maximum : <collez votre brouillon>"
```

(Remplacez `gemma4:12b` par le modèle téléchargé.) La première réponse est plus lente : le modèle se charge en mémoire. Les suivantes arrivent tout de suite.

Étape 5 — Le test « mode avion » (3 min)

Coupez le Wi-Fi (barre des menus), posez une autre question. **Si vous obtenez une réponse, la preuve est faite : rien ne sort de votre Mac.** Rallumez ensuite le Wi-Fi.

Étape 6 — Votre assistant français (4 min)

```
cd ~/ia-locale-kit/modelfiles
ollama create assistant-fr -f assistant-fr.Modelfile
ollama run assistant-fr
```

Le Modelfile du kit part de `gemma4:12b`. Si vous avez téléchargé un autre modèle, remplacez la première ligne `FROM …` par son nom, dans TextEdit (format texte brut).

> **Vous préférez ne jamais ouvrir le Terminal ?** Installez **LM Studio** (https://lmstudio.ai), ouvrez l'onglet de recherche de modèles, tapez `gemma 4`, prenez la version recommandée pour votre Mac (LM Studio signale celles qui ne tiennent pas en mémoire), puis **Load** et discutez. Le test « mode avion » est identique. Le chapitre 3.3 détaille l'interface.

Et maintenant ? Les cas concrets par métier (chapitre 9), la transcription de vos réunions (chapitre 5), puis vos documents (chapitre 6).

Avant de commencer

À qui s'adresse ce guide

Ce guide s'adresse aux indépendants, professions libérales, dirigeants de TPE/PME et curieux techniques qui possèdent un Mac à puce Apple (M1 ou plus récent) et veulent utiliser l'intelligence artificielle **sans envoyer leurs données sur un serveur tiers**. Aucune compétence en programmation n'est requise pour les chapitres 1 à 6 (hors commandes optionnelles) ; les chapitres 7 et 10 utilisent le Terminal, avec des commandes à copier-coller expliquées ligne par ligne.

Ce que vous saurez faire à la fin

- Choisir un Mac (ou vérifier le vôtre) selon la mémoire dont vos usages ont besoin.
- Installer et utiliser **Ollama**, **LM Studio**, **llama.cpp** et **MLX** (mlx-lm), et savoir lequel choisir.
- Choisir un modèle adapté au français et à votre mémoire, et comprendre la quantisation (Q4, Q8, 4-bit MLX…).
- Transcrire des réunions et dictées **entièrement en local** (whisper.cpp, MLX Whisper).
- Interroger vos propres documents (PDF, notes, procédures) avec un **RAG local** (Open WebUI, AnythingLLM, ou le script du kit).
- Automatiser vos tâches répétitives (résumés, comptes rendus, courriels types) avec des scripts et Raccourcis macOS.
- Mettre en place des règles simples de confidentialité et de conformité RGPD.
- Diagnostiquer les problèmes courants (lenteur, mémoire saturée, modèle qui « hallucine »).

Ce que ce guide n'est pas

- Ce n'est **pas** une promesse de remplacer les meilleurs modèles en ligne pour toutes les tâches. Un modèle local de 20 à 35 milliards de paramètres est excellent pour résumer, reformuler, classer, extraire et rédiger ; il reste en dessous des plus grands modèles en ligne pour le raisonnement très long ou les connaissances très pointues.
- Ce n'est **pas** un avis juridique. Le chapitre RGPD donne des repères pratiques ; pour un traitement de données de santé ou un contexte réglementé, validez avec votre DPO ou votre conseil.
- Les noms de modèles et versions évoluent vite. Toutes les références sont **datées (octobre 2026)** et accompagnées de leur source. Le chapitre 4 vous apprend à choisir vous-même quand de nouveaux modèles sortent.

Conventions

- Les commandes à taper dans l'application **Terminal** (Applications › Utilitaires) sont présentées ainsi :

```
ollama --version
```

- `~` désigne votre dossier personnel (par exemple `/Users/marie`).
- « Go » = gigaoctet de mémoire unifiée (RAM) ; « tok/s » = jetons (tokens) générés par seconde, environ 0,75 mot par jeton en anglais, un peu moins en français.

- Les encadrés **À retenir** résument l'essentiel ; les encadrés **Attention** signalent un piège fréquent.

Le kit fourni

Le fichier `kit.zip` contient les scripts, fichiers de configuration, prompts et checklists cités dans ce guide. Chaque fichier commence par un commentaire qui explique son rôle. Décompressez-le dans un dossier de votre choix, par exemple `~/ia-locale-kit`.

> **À retenir** — Rien dans ce guide n'exige de créer un compte en ligne, de payer un abonnement ou de fournir une clé d'API. Une connexion Internet n'est nécessaire que pour télécharger les logiciels et les modèles ; ensuite tout fonctionne hors ligne.

1. Pourquoi faire tourner l'IA en local

Vos données restent chez vous

Quand vous collez un contrat, un dossier patient, un bilan ou un compte rendu de réunion dans un assistant en ligne, ce texte quitte votre machine. Selon le service et l'offre souscrite, il peut être journalisé, conservé un certain temps, examiné à des fins de sécurité, voire servir à l'entraînement. Même avec de bonnes garanties contractuelles, vous ajoutez un **sous-traitant** au sens du RGPD, avec un transfert éventuel hors de l'Union européenne.

En local, le modèle est un fichier sur votre disque ; le calcul a lieu sur votre processeur ; **rien ne sort**. Vous pouvez débrancher le Wi-Fi et vérifier que tout fonctionne encore : c'est le meilleur test de confidentialité qui soit.

Un coût marginal nul

Une fois le Mac acheté, chaque requête est gratuite. Pas de quota, pas de facturation au jeton, pas de hausse de tarif. Pour un usage intensif (transcrire toutes ses réunions, résumer des centaines de documents), la différence devient sensible au bout de quelques mois.

Disponibilité et maîtrise

- Fonctionne dans le train, en zone blanche, chez un client qui interdit les connexions externes.
- Le modèle ne change pas sans vous prévenir : une version que vous avez validée donnera des réponses stables.
- Vous choisissez la licence (Apache 2.0, MIT…) et pouvez l'intégrer à vos outils.

Les limites honnêtes

Critère	Local (Mac 32 Go)	Grand modèle en ligne
Confidentialité	Totale	Dépend du contrat
Coût par requête	0 €	Payant au-delà d'un quota
Qualité rédaction / résumé FR	Très bonne	Excellente
Raisonnement long, mathématiques avancées	Correcte à bonne	Excellente
Connaissances récentes	Date de coupure du modèle	Souvent avec recherche web
Vitesse	20 à 120 tok/s selon modèle et puce	Rapide, dépend du réseau
Installation	15 à 30 minutes	Immédiate

> **À retenir** — La bonne stratégie est souvent **hybride** : tout ce qui contient des données personnelles, clients ou confidentielles reste en local ; les questions génériques sans donnée sensible peuvent aller en ligne si vous le souhaitez.

Pourquoi le Mac est particulièrement adapté

Les puces Apple Silicon partagent une **mémoire unifiée** entre le processeur et la carte graphique. Un Mac de 32 Go peut donc consacrer la majeure partie de cette mémoire au modèle, là où un PC classique est limité par la mémoire de sa carte graphique (souvent 8 à 16 Go). La **bande passante mémoire** (Go/s) détermine en grande partie la vitesse de génération : c'est pourquoi une puce « Max » ou « Ultra » génère plus vite qu'une puce de base, même à mémoire égale.

Apple maintient en outre **MLX**, un cadre de calcul optimisé pour ses puces. Depuis la version 0.40 (fin septembre / début octobre 2026), **Ollama utilise automatiquement MLX** sur Apple Silicon pour les architectures que son moteur MLX prend en charge, et `ollama ps` indique le moteur utilisé dans la colonne RUNNER (source : notes de version Ollama 0.40, résumées par jacar.es).

2. Choisir son Mac et sa mémoire

La règle de calcul

La mémoire nécessaire pour un modèle dépend de son nombre de paramètres et de sa **précision** (quantisation) :

Mémoire ≈ paramètres (en milliards) × octets par paramètre + 1 à 4 Go (contexte et système)

Précision	Octets / paramètre	Modèle 8 B	Modèle 27 B	Modèle 35 B
16 bits (BF16/FP16)	2	~16 Go	~54 Go	~70 Go
8 bits (Q8_0, MLX 8-bit)	~1,06	~8,5 Go	~29 Go	~37 Go
~4,8 bits (Q4_K_M)	~0,6	~5 Go	~16 Go	~21 Go
4 bits (MLX 4-bit)	~0,56	~4,5 Go	~15 Go	~20 Go

Valeurs indicatives ; la taille exacte figure sur la page de chaque modèle.

macOS réserve une partie de la mémoire au système : par défaut, la carte graphique ne peut utiliser qu'environ **les deux tiers à trois quarts** de la mémoire unifiée. Comptez donc qu'un Mac de 16 Go accepte confortablement un modèle de 8 à 10 Go, un Mac de 32 Go un modèle d'environ 20 Go, etc. (voir le chapitre Dépannage pour ajuster cette limite).

Les modèles « MoE » changent la donne

Un modèle à **mélange d'experts** (Mixture of Experts, MoE) comme Qwen 3.6-35B-A3B possède 35 milliards de paramètres au total mais n'en **active qu'environ 3 milliards par jeton**. Il occupe la mémoire d'un modèle de 35 B, mais génère presque à la vitesse d'un modèle de 3 B. C'est pourquoi les MoE dominent les classements Mac en 2026.

Que faire avec quelle mémoire (octobre 2026)

Le tableau suivant reprend les recommandations publiées par l'index LLMCheck (« State of Open-Source Local LLMs: October 2026 »). Les vitesses sont celles publiées par la source (mesures communautaires ou estimations) ; **mesurez toujours sur votre machine** (chapitre 10, script bench.sh).

Mémoire	Exemples de Mac	Modèles conseillés	Vitesse publiée
8 Go	MacBook Air M1/M2 de base	Petits modèles 1–4 B (Gemma 4 E2B/E4B, Qwen3 4B), transcription	Usage limité
16 Go	MacBook Air, Mac mini	GPT-oss 20B ; Gemma 4 12B	45 et 19 tok/s mesurés (Mac mini M6)
24–32 Go	Mac mini, MacBook Air/Pro	Qwen 3.6-35B-A3B ; Gemma 4 26B-A4B	64 et 48 tok/s mesurés (M6) ; 126 tok/s (M5 Max, communauté)
48–64 Go	Mac mini / MacBook Pro « Pro »	Qwen3-Coder-Next ; modèles denses 70 B en 4 bits	~49 et ~6 tok/s (estimés)
128 Go	Mac Studio, MacBook Pro « Max »	Qwen3.8-Flash-Next ; très gros MoE en 2-3 bits	~40-50 tok/s (estimés)
256–512 Go	Mac Studio « Ultra »	Modèles de frontière ouverts	Variable

> **À retenir** — Pour un usage professionnel confortable en 2026, **32 Go** est le meilleur rapport qualité-prix : il fait tourner les meilleurs MoE de taille moyenne (Qwen 3.6-35B-A3B, Gemma 4 26B-A4B), plus Whisper et un modèle d'embeddings en même temps. 16 Go reste utilisable avec GPT-oss 20B ou Gemma 4 12B.

Le disque

Chaque modèle pèse de 2 à 80 Go. Prévoyez **au moins 100 Go libres** pour expérimenter. Un SSD externe Thunderbolt/USB4 convient très bien pour stocker les modèles (chargement un peu plus lent, génération identique une fois le modèle en mémoire).

Vérifier votre Mac

Menu Pomme › **À propos de ce Mac** : vous y lisez la puce (M1, M2 Pro…) et la mémoire. En Terminal :

```
sysctl -n machdep.cpu.brand_string      # puce
sysctl -n hw.memsize | awk '{print $1/1073741824 " Go"}'   # mémoire
df -h ~ | tail -1                       # espace disque libre
```

Le script `kit/scripts/check-mac.sh` fait ces vérifications et vous recommande une taille de modèle.

Et si mon Mac a une puce Intel ?

Les outils fonctionnent en partie sur Mac Intel, mais sans accélération MLX et avec des performances très inférieures. Ce guide vise les puces Apple (M1 et suivantes).

2 bis. Les chiffres : vitesse et mémoire mesurées

Les chiffres de ce chapitre ne sont pas des estimations : ce sont des mesures publiées, avec leur source. Ils servent à **prévoir** ce que votre Mac fera avant d'acheter ou de télécharger.

La règle de mémoire, en une ligne

Taille du modèle en mémoire ≈ nombre de paramètres (en milliards) × bits par poids ÷ 8, plus 10 à 20 % pour le contexte.

Modèle	Q4 (≈ 4,5 bits)	Q8	Mac conseillé
4 milliards (4B)	≈ 2,5 Go	≈ 4,5 Go	8 Go
12B	≈ 7–8 Go	≈ 13 Go	16 Go
24–27B	≈ 15–17 Go	≈ 27 Go	32 Go
35B MoE (3B actifs)	≈ 20–22 Go	≈ 37 Go	32 Go et plus

macOS ne laisse au processeur graphique qu'une partie de la mémoire unifiée (environ deux tiers à trois quarts selon la quantité totale, réglable : chapitre 10.3). Un modèle de 20 Go sur un Mac 24 Go est donc **à la limite** ; il tient confortablement sur 32 Go.

La vitesse dépend surtout de la bande passante mémoire

Mesures `llama-bench` publiées par la communauté llama.cpp (discussion n° 4167, relevé du 31/10/2026) sur un modèle de référence de 7 milliards de paramètres (LLaMA 7B), quantisé en Q4_0. **PP** = lecture du texte fourni (prompt), **TG** = génération de la réponse, en jetons par seconde :

Puce (cœurs GPU)	Bande passante	PP Q4_0 (tok/s)	TG Q4_0 (tok/s)
M1 (8)	68 Go/s	117,96	14,15
M1 Pro (16)	200 Go/s	266,25	36,41
M1 Max (32)	400 Go/s	530,06	61,19
M2 (10)	100 Go/s	179,57	21,91
M2 Max (38)	400 Go/s	671,31	65,95
M3 Pro (18)	150 Go/s	341,67	30,74
M3 Max (40)	400 Go/s	759,70	66,31
M4 (10)	120 Go/s	221,29	24,11
M4 Pro (20)	273 Go/s	439,78	50,74
M4 Max (40)	546 Go/s	885,68	83,06
M5 (10)	154 Go/s	722,79	31,88
M5 Pro (20)	307 Go/s	1 620,64	66,33
M5 Max (40)	614 Go/s	3 219,99	119,92
M5 Ultra (80)	1 228 Go/s	4 944,66	179,10

Source : https://github.com/ggml-org/llama.cpp/discussions/4167 (tableau récapitulatif, valeurs copiées sans arrondi). Les configurations sans mesure publiée (M3 8 cœurs, M5 Max 32 cœurs, etc.) sont omises.

Comment lire ce tableau

- **La génération (TG) suit la bande passante.** Un M4 Pro (273 Go/s) génère environ deux fois plus vite qu'un M4 (120 Go/s) ; à bande passante égale (M1 Max, M2 Max, M3 Max à 400 Go/s), les vitesses sont proches.
- **Ramenez à votre modèle.** La vitesse de génération baisse à peu près en proportion de la mémoire lue à chaque jeton. Un modèle **dense** de 24B (≈ 3,5 fois plus lourd que le 7B) génère environ 3,5 fois moins vite. Un **MoE** comme `qwen3.6:35b` ne lit qu'environ 3 milliards de paramètres actifs par jeton : il génère **plus vite qu'un 7B dense**, mais il doit tenir **entièrement** en mémoire.
- **Le seuil de confort de lecture** se situe autour de 10 à 15 tok/s ; au-dessus de 30 tok/s, la réponse s'affiche plus vite qu'on ne la lit.
- **La lecture du prompt (PP)** compte pour le RAG et les longs documents : un M5 lit un contexte de 8 000 jetons en une dizaine de secondes, un M1 en plus d'une minute.

Mesurer votre propre Mac

```
bash ~/ia-locale-kit/scripts/bench.sh gemma4:12b
```

Le script affiche la vitesse de génération et le temps de chargement. Lancez-le deux fois : la première mesure inclut le chargement depuis le disque.

> **À retenir** — Pour choisir un Mac destiné à l'IA locale : d'abord la **mémoire** (elle décide quels modèles tiennent), ensuite la **bande passante** (elle décide de la vitesse). Une puce Pro avec 48 Go est souvent un meilleur investissement qu'une puce de base avec 32 Go.

3. Installer les outils

Quatre outils dominent l'IA locale sur Mac. Ils ne s'excluent pas : la plupart des utilisateurs installent **Ollama** (le moteur de fond) et **LM Studio** (l'interface graphique), puis ajoutent mlx-lm ou llama.cpp pour des besoins précis.

Outil	Pour qui	Interface	API locale	Formats
Ollama	Tout le monde, usage quotidien et automatisation	App + Terminal	`http://localhost:11434` (native + compatible OpenAI `/v1`)	Bibliothèque Ollama (MLX ou GGUF selon le modèle)
LM Studio	Ceux qui préfèrent une interface graphique	App complète + CLI `lms`	`http://localhost:1234/v1` (compatible OpenAI)	GGUF et MLX
llama.cpp	Techniciens, contrôle fin, serveurs	Terminal + interface web intégrée	`llama-server`, port 8080 par défaut	GGUF
mlx-lm	Modèles MLX de Hugging Face, ajustement fin (fine-tuning)	Terminal / Python	`mlx_lm.server`, port 8080 par défaut	MLX (safetensors)

> **Attention** — llama-server, mlx_lm.server et Open WebUI utilisent tous le port **8080** par défaut. Si vous en lancez deux, changez le port de l'un (`--port 8081`).

3.1 Prérequis : Homebrew (optionnel mais pratique)

Homebrew est le gestionnaire de paquets le plus répandu sur Mac. Instructions officielles : https://brew.sh. Après installation :

```
brew --version
brew install ffmpeg        # utile pour l'audio (chapitre 5)
```

3.2 Ollama

Installation : téléchargez l'application sur https://ollama.com/download, glissez-la dans Applications et lancez-la. Une icône apparaît dans la barre des menus ; le service écoute alors sur `localhost:11434`. (Alternative : `brew install ollama`, puis `ollama serve` ou `brew services start ollama`.)

Premiers pas :

```
ollama --version
ollama pull qwen3.6:35b      # télécharge le modèle (~20+ Go, une seule fois)
ollama run qwen3.6:35b       # discussion interactive ; /bye pour quitter
ollama list                  # modèles installés
ollama ps                    # modèles chargés en mémoire, moteur (RUNNER) utilisé
ollama rm nom:tag            # supprimer un modèle
```

Sur un Mac 16 Go, remplacez par `gemma4:12b` ou `gpt-oss:20b`. Sur 8 Go, `gemma4:e4b` ou `qwen3:4b`.

Une question ponctuelle, sans mode interactif :

```
ollama run gemma4:12b "Résume en 3 points : $(cat note.txt)"
```

L'API (pour vos scripts) :

```
curl http://localhost:11434/api/chat -d '{
  "model": "gemma4:12b",
  "messages": [{"role": "user", "content": "Bonjour, qui es-tu ?"}],
  "stream": false
}'
```

Ollama expose aussi une API compatible OpenAI sur `http://localhost:11434/v1`, ce qui permet de brancher la plupart des logiciels prévus pour OpenAI en changeant simplement l'adresse (et en mettant n'importe quelle valeur comme clé).

Personnaliser avec un Modelfile (exemple fourni : `kit/modelfiles/assistant-fr.Modelfile`) :

```
FROM gemma4:12b
PARAMETER temperature 0.4
PARAMETER num_ctx 16384
SYSTEM """Tu es un assistant professionnel francophone. Réponds en français,
de façon concise et structurée. Si tu ne sais pas, dis-le."""
```

```
ollama create assistant-fr -f assistant-fr.Modelfile
ollama run assistant-fr
```

Réglages utiles (variables d'environnement, à définir avant de lancer `ollama serve`, ou via `launchctl setenv` pour l'application) :

Variable	Rôle
`OLLAMA_HOST`	Adresse d'écoute (laisser `127.0.0.1` pour rester local)
`OLLAMA_MODELS`	Dossier de stockage des modèles (ex. SSD externe)
`OLLAMA_KEEP_ALIVE`	Durée de maintien en mémoire après usage (ex. `10m`, `-1` = toujours)
`OLLAMA_CONTEXT_LENGTH`	Contexte par défaut

> **Attention** — Certains modèles de la bibliothèque portent la mention **cloud** (par exemple certaines variantes de très grands modèles). Ils s'exécutent sur les serveurs d'Ollama, pas sur votre Mac. Pour rester 100 % local, n'utilisez que des modèles **sans** l'étiquette « cloud » et vérifiez avec `ollama ps` qu'ils occupent bien votre mémoire.

3.3 LM Studio

Installation : https://lmstudio.ai, téléchargez la version Mac (Apple Silicon). L'application propose :

- un onglet **Discover** pour chercher des modèles (filtrez par « MLX » pour les puces Apple, ou « GGUF ») ; LM Studio indique si le modèle tient dans votre mémoire ;
- un onglet **Chat** avec gestion de conversations et pièces jointes (documents) ;
- un onglet **Developer** pour lancer un serveur local compatible OpenAI (port 1234).

La ligne de commande `lms` (installée avec l'application) :

```
lms ls                      # modèles téléchargés
lms get <nom-du-modèle>     # télécharger
lms load <nom-du-modèle>    # charger en mémoire
lms server start            # démarrer le serveur local
lms ps                      # modèles chargés
```

> **À retenir** — LM Studio est idéal pour **comparer** des modèles : téléchargez-en deux ou trois, posez la même question, gardez le meilleur. Ollama est idéal pour **automatiser** : il tourne en fond et répond aux scripts.

3.4 llama.cpp

C'est le moteur d'origine du format GGUF, utilisé en interne par de nombreux outils.

```
brew install llama.cpp
# Télécharge depuis Hugging Face et lance un serveur avec interface web :
llama-server -hf ggml-org/gemma-3-12b-it-GGUF:Q4_K_M --port 8080
```

Ouvrez ensuite http://localhost:8080 dans votre navigateur. Le suffixe `:Q4_K_M` choisit la quantisation (chapitre 4). Pour une utilisation en ligne de commande : `llama-cli -hf <dépôt>:<quant>`.

Options fréquentes : `-c 16384` (taille du contexte), `-ngl 99` (tout sur le GPU, valeur par défaut sur Mac récents), `--host 127.0.0.1`.

3.5 MLX et mlx-lm

MLX est le cadre d'Apple ; **mlx-lm** permet de faire tourner des modèles de texte au format MLX, publiés notamment par la communauté `mlx-community` sur Hugging Face.

```
python3 -m venv ~/venvs/mlx && source ~/venvs/mlx/bin/activate
pip install -U mlx-lm
mlx_lm.generate --model mlx-community/<modèle>-4bit --prompt "Bonjour"
mlx_lm.chat --model mlx-community/<modèle>-4bit
mlx_lm.server --model mlx-community/<modèle>-4bit --port 8081
```

Remplacez `<modèle>` par un nom exact copié depuis https://huggingface.co/mlx-community (par exemple une variante 4-bit de Qwen ou Gemma). `mlx_lm.convert --model <dépôt-hf> -q` permet de convertir et quantiser vous-même un modèle.

Quand choisir mlx-lm plutôt qu'Ollama ? D'après le comparatif localaiscope (octobre 2026), Ollama exécute déjà en MLX les familles qu'il connaît (Qwen 3.5/3.6/3.8, Gemma 4, etc.). mlx-lm reste utile pour les modèles que le moteur MLX d'Ollama ne prend pas en charge (le comparatif

cite gpt-oss, Mistral Small 24B, Phi-4, Granite 4.2…), pour choisir une compression précise et pour l'ajustement fin.

3.6 Tableau de décision

Votre besoin	Outil
« Je veux discuter avec une IA privée, simplement »	LM Studio
« Je veux qu'un script ou une app utilise l'IA »	Ollama
« Je veux interroger mes documents »	Open WebUI ou AnythingLLM + Ollama (chapitre 6)
« Je veux le dernier modèle MLX publié hier sur Hugging Face »	mlx-lm ou LM Studio
« Je veux un serveur pour toute l'équipe sur un Mac mini »	Ollama ou llama-server (chapitre 9)

4. Choisir ses modèles

Lire le nom d'un modèle

Prenons `Qwen3.6-35B-A3B-Instruct-Q4_K_M.gguf` :

Fragment	Signification
Qwen3.6	Famille et version (éditeur : équipe Qwen d'Alibaba)
35B	35 milliards de paramètres au total
A3B	~3 milliards de paramètres **actifs** par jeton (modèle MoE)
Instruct / it	Version ajustée pour suivre des consignes (celle qu'il vous faut)
Q4_K_M	Quantisation GGUF ~4,8 bits, variante « moyenne »
.gguf	Format llama.cpp (Ollama, LM Studio) ; un dossier « -4bit » sur mlx-community = format MLX

La quantisation, simplement

Un modèle est publié en 16 bits par paramètre. **Quantiser**, c'est arrondir ces nombres sur moins de bits pour réduire la taille et accélérer le calcul, avec une petite perte de qualité.

Quantisation	Taille vs 16 bits	Perte de qualité	Usage conseillé
Q8_0 / MLX 8-bit	~53 %	Quasi nulle	Si la mémoire le permet
Q6_K	~41 %	Très faible	Bon compromis sur 64 Go et plus
Q5_K_M	~36 %	Faible	Bon compromis
Q4_K_M / MLX 4-bit	~30 %	Faible à modérée	**Choix par défaut**
Q3_K_M	~25 %	Visible (erreurs, français moins fluide)	Uniquement pour faire tenir un gros modèle
Q2_K / IQ2	~18 %	Forte	Expérimentation

> **À retenir** — À mémoire égale, un **modèle plus grand en Q4** est généralement meilleur qu'un **modèle plus petit en Q8**. Ne descendez sous Q4 qu'en connaissance de cause.

Les suffixes « K_S / K_M / K_L » désignent des variantes petite/moyenne/grande ; les quantisations « IQ » (i-quants) sont plus fines à taille égale mais parfois plus lentes. Les modèles quantisés par des tiers reconnus (ggml-org, bartowski, unsloth, lmstudio-community, mlx-community) sont un gage de sérieux.

Le contexte

Le **contexte** est la quantité de texte que le modèle « voit » à la fois (question + documents + réponse), en jetons. 1 page A4 ≈ 600 à 800 jetons en français. Un contexte de 16 384 jetons ≈ 20

pages. Plus le contexte est grand, plus il consomme de mémoire (cache « KV ») : sur 16 Go, restez vers 8 000 à 16 000 jetons ; sur 32 Go, 32 000 est raisonnable avec un MoE.

Sélection d'octobre 2026

Les modèles ci-dessous figurent dans la bibliothèque Ollama (consultée le 30/10/2026) et/ou sur Hugging Face, et la plupart sont cités dans l'index LLMCheck d'octobre 2026. **Vérifiez toujours la licence** sur la page du modèle avant un usage commercial.

Généralistes (rédaction, résumé, analyse)

Modèle	Commande Ollama	Mémoire mini	Remarques
Qwen 3.6 35B-A3B (MoE)	`ollama run qwen3.6:35b`	24 Go	N°1 de l'index LLMCheck au 24/10/2026 pour les Mac ≤ 32 Go ; Apache 2.0 ; très rapide
Gemma 4 26B-A4B (MoE)	`ollama run gemma4:26b`	18 Go	Google, Apache 2.0 ; vision
Gemma 4 12B	`ollama run gemma4:12b`	16 Go Mac	Bon français, vision ; idéal 16 Go
GPT-oss 20B	`ollama run gpt-oss:20b`	16 Go Mac	Modèle ouvert d'OpenAI ; raisonnement
Qwen3.8 27B (dense)	`ollama run qwen3.8:27b`	19 Go+	Plus lent qu'un MoE, très solide
Mistral Small 3.2 24B	`ollama run mistral-small3.2:24b`	24 Go	Éditeur français ; vision
Ministral 3 (3/8/14B)	`ollama run ministral-3:8b`	8–16 Go	Mistral AI, pour petites machines
Gemma 4 E4B / E2B	`ollama run gemma4:e4b`	8 Go	Très légers

Code

Modèle	Commande	Remarques
Qwen3-Coder 30B (MoE)	`ollama run qwen3-coder:30b`	Agents de code, long contexte
Devstral Small 2 24B	`ollama run devstral-small-2:24b`	Mistral AI, édition multi-fichiers
Qwen3-Coder-Next	`ollama run qwen3-coder-next`	48–64 Go conseillés

Vision et documents scannés

Modèle	Commande	Remarques
Qwen3-VL	`ollama run qwen3-vl:8b`	Description d'images, captures d'écran
GLM-OCR	`ollama run glm-ocr`	Lecture de documents complexes (OCR)
Gemma 4 (toutes tailles)	voir ci-dessus	Multimodal natif

Embeddings (indispensables pour le RAG, chapitre 6)

Modèle	Commande	Remarques
BGE-M3	`ollama pull bge-m3`	Multilingue, très bon en français
Qwen3-Embedding	`ollama pull qwen3-embedding:0.6b`	0.6b / 4b / 8b
nomic-embed-text-v2-moe	`ollama pull nomic-embed-text-v2-moe`	Multilingue, léger
EmbeddingGemma	`ollama pull embeddinggemma`	300 M paramètres

Équivalents MLX (mlx-lm, LM Studio)

Exemples de dépôts publiés par mlx-community (vérifiés le 30/10/2026) : `mlx-community/Qwen3.6-35B-A3B-4bit`, `mlx-community/gemma-4-26b-a4b-it-4bit`, `mlx-community/gemma-4-e4b-it-8bit`, `mlx-community/gemma-4-e2b-it-4bit`.

> **Attention** — Sur Hugging Face, de nombreux dépôts sont des variantes modifiées par des tiers (« abliterated », « uncensored », fusions…). Pour un usage professionnel, préférez les versions officielles de l'éditeur ou les quantisations directes des organisations reconnues citées plus haut.
>
> **Attention** — N'utilisez pas les modèles marqués **cloud** dans la bibliothèque Ollama (ex. certaines variantes de très grands modèles) si votre objectif est la confidentialité : ils ne tournent pas sur votre Mac.

Tester un modèle en 10 minutes

Le fichier `kit/prompts/banc-essai-fr.md` contient 8 questions types (résumé, courriel, extraction de tableau, reformulation juridique simple, traduction, raisonnement, refus de répondre quand l'information manque, consigne de format). Posez-les à deux ou trois modèles et notez sur 5 : exactitude, qualité du français, respect du format, vitesse ressentie. Le script `kit/scripts/bench.sh` mesure la vitesse (tok/s) via l'API Ollama.

Les « modèles qui réfléchissent »

Plusieurs modèles (Qwen 3.x, GPT-oss, Gemma 4…) ont un mode **thinking** : ils produisent un raisonnement intermédiaire avant la réponse. C'est plus fiable pour les problèmes complexes, mais plus lent. Dans Ollama, l'option `--think=false` (ou `"think": false` dans l'API) désactive ce mode pour les tâches simples (reformuler, résumer), lorsque le modèle le permet.

5. Transcrire ses réunions en local

La transcription est souvent **le premier gain concret** : une réunion d'une heure, transcrite et résumée en quelques minutes, sans qu'aucun enregistrement ne quitte le Mac.

5.1 Les deux options

Outil	Installation	Atout
whisper.cpp	`brew install whisper.cpp` (ou compilation depuis GitHub)	Léger, rapide, sans Python
MLX Whisper	`pip install mlx-whisper`	Optimisé MLX, modèles Hugging Face mlx-community

Les deux utilisent les modèles **Whisper** (OpenAI, licence MIT), qui gèrent très bien le français. Le modèle **large-v3-turbo** offre le meilleur compromis vitesse/précision ; `small` ou `base` suffisent pour une dictée claire sur un petit Mac.

5.2 Préparer l'audio

whisper.cpp attend un fichier WAV 16 kHz mono 16 bits (indiqué dans sa documentation). Convertissez n'importe quel enregistrement (m4a d'iPhone, mp3, vidéo) avec ffmpeg :

```
ffmpeg -i reunion.m4a -ar 16000 -ac 1 -c:a pcm_s16le reunion.wav
```

5.3 whisper.cpp

```
# Télécharger un modèle (une fois) depuis le dépôt whisper.cpp :
git clone https://github.com/ggml-org/whisper.cpp
cd whisper.cpp
sh ./models/download-ggml-model.sh large-v3-turbo
# Transcrire en français, sortie texte + sous-titres :
whisper-cli -m models/ggml-large-v3-turbo.bin -l fr -f ~/reunion.wav -otxt -osrt
```

(Si vous avez compilé vous-même, le binaire est `./build/bin/whisper-cli`.) Les fichiers `reunion.wav.txt` et `reunion.wav.srt` sont créés à côté de l'audio.

5.4 MLX Whisper

```
source ~/venvs/mlx/bin/activate
pip install -U mlx-whisper
mlx_whisper ~/reunion.m4a --model mlx-community/whisper-large-v3-turbo --language fr --
output-format txt
```

MLX Whisper s'appuie sur ffmpeg pour lire directement les m4a/mp3.

5.5 De la transcription au compte rendu

Le script `kit/scripts/transcrire-et-resumer.sh` enchaîne : conversion ffmpeg → transcription → envoi du texte à Ollama avec le prompt `kit/prompts/compte-rendu.md`, qui produit :

1. une synthèse en 5 lignes ;
2. les décisions prises ;
3. les actions (qui, quoi, quand) sous forme de tableau ;
4. les points ouverts.

```
~/ia-locale-kit/scripts/transcrire-et-resumer.sh ~/Desktop/reunion.m4a
```

> **Attention** — Pour une réunion de plus d'une heure, la transcription dépasse souvent le contexte du modèle. Le script découpe automatiquement le texte en blocs, résume chaque bloc, puis fusionne les résumés (technique « map-reduce »).

5.6 Bonnes pratiques

- **Informez les participants** que la réunion est enregistrée et transcrite, et pour quel usage (obligation de transparence).
- Supprimez l'audio dès que la transcription est validée, si vous n'en avez pas besoin (minimisation).
- Whisper ne distingue pas les intervenants (« diarisation ») : pour cela, des outils comme pyannote existent mais demandent une installation plus technique ; une alternative simple est de demander au modèle de proposer une attribution des prises de parole, à vérifier.
- Relisez les noms propres, chiffres et termes techniques : ce sont les erreurs les plus fréquentes. Vous pouvez donner à whisper.cpp une liste de vocabulaire via l'option `--prompt "Noms : Dupont, Bayonne, AMDEC…"`.

5.7 Dictée au quotidien

macOS propose une **dictée intégrée** (Réglages › Clavier › Dictée) qui, sur Apple Silicon, traite de nombreuses langues sur l'appareil. Pour des textes longs ou du vocabulaire métier, un raccourci qui enregistre puis appelle Whisper (chapitre 7) donne souvent de meilleurs résultats.

6. Interroger ses documents : le RAG local

Le **RAG** (Retrieval-Augmented Generation, « génération augmentée par recherche ») permet de poser des questions à vos propres documents : contrats, procédures, notes, jurisprudence interne, documentation produit. Le modèle ne « connaît » pas vos fichiers ; on lui fournit, à chaque question, les passages les plus pertinents.

6.1 Le principe en 4 étapes

1. **Découper** les documents en passages (300 à 800 mots, avec chevauchement).
2. **Vectoriser** chaque passage avec un modèle d'embeddings (`bge-m3`, `qwen3-embedding`…) : on obtient une empreinte numérique du sens.
3. À chaque question, **retrouver** les passages dont l'empreinte est la plus proche.
4. **Générer** la réponse en donnant au modèle la question + ces passages, avec la consigne de citer ses sources et de dire « je ne sais pas » si l'information est absente.

> **À retenir** — La qualité d'un RAG dépend surtout de l'étape 1 (documents propres, bien découpés) et du modèle d'embeddings, plus que du modèle de génération.

6.2 Option clé en main : Open WebUI

Open WebUI est une interface web open source qui se connecte à Ollama et intègre la gestion de documents (« Knowledge »).

```
python3.12 -m venv ~/venvs/webui && source ~/venvs/webui/bin/activate
pip install open-webui
open-webui serve            # puis http://localhost:8080
```

La documentation officielle recommande Python 3.11 ou 3.12 pour l'installation par pip ; une installation Docker est aussi proposée. Au premier lancement, créez le compte administrateur (local). Ensuite :

1. Réglages admin › Connexions : vérifiez l'adresse d'Ollama (`http://localhost:11434`).
2. Réglages admin › Documents : choisissez le moteur d'embeddings « Ollama » et le modèle `bge-m3`.
3. Espace de travail › Connaissances : créez une base, déposez vos PDF/Word/textes.
4. Dans une discussion, tapez `#` pour attacher la base, puis posez votre question.

6.3 Autres options graphiques

- **AnythingLLM** (application Mac, https://anythingllm.com) : espaces de travail par dossier, se branche sur Ollama ou LM Studio.
- **LM Studio** : déposer un document dans une discussion active une recherche dans ce document (pratique pour un fichier ponctuel, moins pour une base entière).

6.4 Option « je maîtrise tout » : le script du kit

`kit/rag/rag_local.py` est un RAG minimal (≈ 150 lignes, Python standard + `pypdf`) qui n'utilise **que** l'API locale d'Ollama :

```
python3 -m venv ~/venvs/rag && source ~/venvs/rag/bin/activate
pip install pypdf
ollama pull bge-m3 && ollama pull gemma4:12b
python rag_local.py index ~/Documents/Procedures      # crée index.json
python rag_local.py ask "Quel est le délai de préavis prévu ?"
```

La réponse indique les fichiers et passages utilisés. L'index est un simple fichier JSON stocké à côté du script : facile à sauvegarder, à chiffrer ou à supprimer.

6.5 Préparer ses documents

- Les PDF scannés (images) doivent d'abord être OCRisés : l'aperçu macOS sélectionne déjà le texte des images (Live Text), et `ocrmypdf` (`brew install ocrmypdf`) ajoute une couche texte en lot. Pour les documents complexes, un modèle comme `glm-ocr` peut aider.
- Supprimez doublons et versions obsolètes : le RAG citera sinon une ancienne version.
- Nommez clairement les fichiers (`2026-03_contrat-type-honoraires.pdf`) : le nom est repris dans les citations.
- Séparez les bases par niveau de confidentialité (ex. une base « procédures internes », une base par client si nécessaire).

6.6 Limites à connaître

- Un RAG peut **retrouver le mauvais passage** et répondre avec assurance : exigez les citations et vérifiez-les.
- Les questions de synthèse globale (« résume tous nos contrats ») fonctionnent mal : le RAG ne voit que quelques passages à la fois.
- Les tableaux complexes se découpent mal : convertissez-les en texte structuré si possible.

7. Automatiser : Raccourcis, scripts et API

7.1 Un service « toujours prêt »

Laissez Ollama tourner en fond (l'application se lance à l'ouverture de session si vous le souhaitez). Avec `OLLAMA_KEEP_ALIVE=30m`, le modèle reste chargé une demi-heure après la dernière requête : les réponses suivantes démarrent instantanément.

7.2 Raccourcis macOS

L'application **Raccourcis** peut appeler l'API locale avec l'action « Obtenir le contenu de l'URL » :

1. Action « Recevoir » : le texte sélectionné (raccourci disponible dans le menu Services).
2. Action « Obtenir le contenu de l'URL » : URL `http://localhost:11434/api/generate`, méthode POST, corps JSON : `model` = `gemma4:12b`, `prompt` = « Reformule ce texte de façon professionnelle : » + entrée, `stream` = faux (booléen).
3. Action « Obtenir la valeur de response dans le dictionnaire ».
4. Action « Copier dans le presse-papiers » (ou « Remplacer le texte »).

Plus simple encore : l'action « Exécuter un script shell » avec le script du kit :

```
~/ia-locale-kit/scripts/ia.sh "Reformule de façon professionnelle :"
```

`ia.sh` lit le texte sur l'entrée standard, appelle Ollama et renvoie la réponse. Attribuez un raccourci clavier dans Réglages Système › Clavier › Raccourcis › Services.

Idées de raccourcis : reformuler un courriel, corriger l'orthographe, traduire FR↔EN/ES, résumer une page, extraire les dates et montants d'un texte, transformer des notes en compte rendu.

7.3 Traiter un dossier entier

```
for f in ~/Desktop/a-resumer/*.txt; do
  ~/ia-locale-kit/scripts/ia.sh "Résume en 5 puces :" < "$f" > "${f%.txt}.resume.md"
done
```

7.4 Python en 10 lignes

```
import json, urllib.request
def ia(prompt, model="gemma4:12b"):
    data = json.dumps({"model": model, "prompt": prompt, "stream": False}).encode()
    req = urllib.request.Request("http://localhost:11434/api/generate", data,
                                 {"Content-Type": "application/json"})
    return json.load(urllib.request.urlopen(req))["response"]
print(ia("Donne trois idées de titre pour une note sur le RGPD."))
```

Pour les bibliothèques prévues pour OpenAI : `base_url="http://localhost:11434/v1"`, `api_key="ollama"` (valeur quelconque).

7.5 Sorties structurées

Pour extraire des données (factures, formulaires), demandez du JSON avec un schéma. L'API d'Ollama accepte le paramètre `format` (la valeur `"json"` ou un schéma JSON complet) :

```
curl -s http://localhost:11434/api/chat -d '{
 "model":"gemma4:12b","stream":false,
 "messages":[{"role":"user","content":"Facture n°2026-114 du 12/10/2026, total 1 250,00 EUR
TTC, client Dupont SARL"}],
 "format":{"type":"object","properties":{"numero":{"type":"string"},"date":
{"type":"string"},"total_ttc":{"type":"number"},"client":{"type":"string"}},"required":
["numero","date","total_ttc","client"]}
}' | python3 -c "import json,sys; print(json.load(sys.stdin)['message']['content'])"
```

7.6 Outils de développement

Les éditeurs et agents de code qui acceptent un point d'accès compatible OpenAI ou Ollama (extensions VS Code comme Continue, Zed, etc.) peuvent utiliser `qwen3-coder:30b` ou `devstral-small-2:24b` en local. Attendez-vous à une qualité inférieure aux meilleurs services en ligne sur les tâches longues, mais suffisante pour l'autocomplétion, l'explication de code et les petites modifications.

8. RGPD, secret professionnel et bonnes pratiques

> **Avertissement** — Ce chapitre est une aide pratique, pas un conseil juridique. Pour une situation précise, consultez votre DPO, votre ordre professionnel ou un avocat spécialisé. Les références citées sont à vérifier dans leur version en vigueur (Légifrance, cnil.fr).

8.1 Ce que le local change… et ne change pas

Ce qui change : en local, il n'y a **pas de transfert** des données à un fournisseur d'IA, donc pas de sous-traitant IA à encadrer contractuellement (article 28 du RGPD), pas de transfert hors UE (chapitre V du RGPD), pas de réutilisation de vos données pour entraîner un modèle tiers.

Ce qui ne change pas : vous restez **responsable de traitement**. Les principes du RGPD s'appliquent toujours :

Principe (art. 5 RGPD)	Traduction pratique
Finalité déterminée	Écrire *pourquoi* on utilise l'IA (ex. « rédaction de projets de courriers »)
Minimisation	Ne donner au modèle que ce qui est nécessaire ; pseudonymiser quand c'est possible
Exactitude	Relire et vérifier toute production avant usage
Limitation de conservation	Purger transcriptions, historiques et index RAG selon une durée définie
Sécurité (art. 32)	Chiffrement du disque, sessions verrouillées, sauvegardes chiffrées

8.2 Professions soumises au secret

- **Avocats** : le secret professionnel est d'ordre public (art. 66-5 de la loi n° 71-1130 du 31 décembre 1971). Le traitement local évite de confier des pièces à un prestataire extérieur ; les recommandations ordinales et les guides du CNB sur l'IA générative sont à consulter.
- **Santé** : le secret médical (art. L.1110-4 du Code de la santé publique) et les règles d'hébergement des données de santé (HDS, art. L.1111-8) s'appliquent dès qu'on confie des données à un hébergeur tiers. Un traitement **sur le poste du professionnel**, sans hébergement externe, sort de ce schéma — mais les sauvegardes en ligne, elles, doivent être examinées.
- **Experts-comptables, notaires** : obligations de secret et de confidentialité propres à chaque profession ; même logique.

8.3 Le règlement européen sur l'IA (AI Act)

Le règlement (UE) 2024/1689 s'applique progressivement. Pour un utilisateur professionnel (« déployeur ») d'outils d'IA généralistes, les points pratiques sont notamment :

- **Maîtrise de l'IA** (article 4, applicable depuis le 2 février 2025) : s'assurer que les personnes qui utilisent ces outils ont un niveau de compréhension suffisant. Ce guide et une courte formation interne documentée y contribuent.
- **Transparence** : informer lorsqu'un contenu est généré ou manipulé par IA dans les cas prévus par le texte (article 50).
- Les usages dits « à haut risque » (annexe III : recrutement, accès à certains services essentiels, etc.) sont soumis à des obligations renforcées : évitez de les mettre en place sans analyse.

8.4 Sécuriser le Mac

1. **FileVault activé** (Réglages › Confidentialité et sécurité). Indispensable.
2. Verrouillage automatique rapide, mot de passe fort, compte non administrateur au quotidien.
3. Ollama, LM Studio et Open WebUI écoutent par défaut sur `127.0.0.1` (votre Mac seulement). **Ne les exposez pas** sur le réseau sans authentification ; si vous partagez un serveur d'équipe (chapitre 9), restreignez par pare-feu et réseau local.
4. Mises à jour régulières de macOS et des outils.
5. Sauvegardes Time Machine sur disque **chiffré**.
6. Téléchargez les modèles depuis des sources reconnues (bibliothèque Ollama, éditeurs officiels, organisations connues sur Hugging Face).

8.5 Documenter (registre et charte)

Le kit contient :

- `kit/rgpd/checklist-rgpd.md` : 25 points à cocher ;
- `kit/rgpd/fiche-registre.md` : modèle de fiche pour le registre des traitements (art. 30) ;
- `kit/rgpd/charte-usage-ia.md` : modèle de charte interne d'usage (une page), à adapter.

8.6 Le piège des « hybrides »

Beaucoup d'applications « avec IA » envoient discrètement les données vers un service en ligne. Vérifiez les réglages : modèles « cloud » d'Ollama, extensions de navigateur, correcteurs en ligne, synchronisation iCloud de dossiers contenant transcriptions ou index. Le test simple : coupez le Wi-Fi ; si tout fonctionne encore, le traitement est local.

9. Six cas concrets, pas à pas

Chaque cas suit la même fiche : **situation**, **outil et modèle**, **prompt exact** (fichier du kit), **document d'essai** fourni dans `kit/exemples/` (entièrement fictif), **ce que vous devez obtenir** et **grille de contrôle**. Faites d'abord l'essai avec le document fictif : vous saurez si votre installation est au niveau **avant** d'y mettre un vrai dossier.

> **Principe commun** — l'IA prépare, vous décidez. Dans chaque cas, la grille de contrôle désigne ce que **vous** devez vérifier. Comptez 2 à 5 minutes de relecture : c'est ce temps, et non la génération, qui fait la qualité.

9.1 Avocat : chronologie des faits à partir de pièces

Situation. Un dossier de licenciement : quatre pièces (courriels, lettre de convocation, attestation). Vous voulez une chronologie sourcée pour préparer les conclusions.

Outil. Ollama + `assistant-fr` (ou Open WebUI avec une base « dossier » si les pièces sont nombreuses : chapitre 6). Modèle : `qwen3.6:35b` (32 Go) ou `gemma4:12b` (16 Go).

Commande.

```
cd ~/ia-locale-kit
ollama run assistant-fr "$(cat prompts/chronologie.md)

$(cat exemples/avocat-pieces.txt)"
```

Ce que vous devez obtenir. Un tableau Date / Fait / Source avec **7 lignes**, une par date présente dans le document d'essai, dont une signalée comme approximative (« courant mars 2026 »).

Grille de contrôle. - ☐ Chaque date existe dans la pièce citée (contrôle par recherche Cmd+F). - ☐ Aucune date déduite (« le lendemain », « deux jours plus tard ») n'a été convertie en date précise sans le signaler. - ☐ Les faits sont rapportés, pas qualifiés juridiquement.

Si le résultat est mauvais. Le modèle invente une date : passez à un modèle plus grand, ou découpez les pièces une par une puis fusionnez les tableaux.

9.2 Expert-comptable : extraire une facture vers un tableur

Situation. Des factures fournisseurs en PDF texte à saisir. Objectif : une ligne de tableur par facture.

Outil. API Ollama en **sortie structurée** (le modèle est contraint à produire un JSON valide), script du kit. Modèle : `gemma4:12b` suffit.

Commande.

```
python3 scripts/extraire_facture.py exemples/facture-fictive.txt >> factures.csv
```

Ce que vous devez obtenir. Une ligne CSV : `2026-0412;Atelier Dupont SARL;2026-10-06;1250.00;250.00;1500.00` (n° ; fournisseur ; date ; HT ; TVA ; TTC).

Grille de contrôle. - ☐ Le script **recalcule** HT + TVA = TTC et signale `ECART` si la somme ne tombe pas juste : un modèle de langage n'est pas une calculatrice. - ☐ La date est au format AAAA-MM-JJ. - ☐ Pour les PDF scannés (images), passer d'abord par l'OCR (`glm-ocr`, chapitre 4) et contrôler les montants.

9.3 Notaire : comparer deux versions d'un acte

Situation. Le confrère renvoie un projet modifié sans marques de révision.

Outil. D'abord une comparaison **textuelle exacte** (`diff`, ou Word › Révision › Comparer), puis l'IA pour **expliquer** les écarts. Modèle : `qwen3.6:35b` ou `mistral-small3.2:24b`.

Commande.

```
diff exemples/acte-v1.txt exemples/acte-v2.txt
ollama run assistant-fr "$(cat prompts/comparer-versions.md)
VERSION A :
$(cat exemples/acte-v1.txt)
VERSION B :
$(cat exemples/acte-v2.txt)"
```

Ce que vous devez obtenir. Trois modifications dans le document d'essai : le prix (185 000 € devenu 182 500 €), le délai de la condition suspensive (45 jours devenus 60 jours) et une clause de répartition de la taxe foncière **ajoutée**.

Grille de contrôle. - ☐ Le nombre d'écarts annoncés par l'IA est égal à celui de `diff`. S'il en manque un, c'est le `diff` qui fait foi. - ☐ L'explication ne contient aucun avis sur la validité de la clause.

9.4 Kinésithérapeute ou médecin : dictée → compte rendu structuré

Situation. Deux minutes de dictée après une séance ; vous voulez un compte rendu au format SOAP (Subjectif, Objectif, Analyse, Plan).

Outil. `whisper.cpp` (chapitre 5) pour la transcription, puis `prompts/compte-rendu-medical.md`. Tout reste sur le Mac.

Commande.

```
bash scripts/transcrire-et-resumer.sh dictee.m4a compte-rendu-medical
```

Pour un essai sans micro, utilisez la transcription fictive : `ollama run assistant-fr "$(cat prompts/compte-rendu-medical.md) $(cat exemples/dictee-kine.txt)"`.

Ce que vous devez obtenir. Les quatre rubriques SOAP remplies ; l'échelle de douleur (EVA 6/10 puis 3/10) et le nombre de séances restantes (4) repris **à l'identique**.

Grille de contrôle. - ☐ Aucun élément clinique ajouté qui n'était pas dicté. - ☐ Les chiffres (EVA, amplitudes, nombre de séances) correspondent à la dictée. - ☐ Le fichier audio est supprimé après

validation si vous ne devez pas le conserver.

9.5 PME : compte rendu de réunion avec actions

Situation. Réunion de chantier ou comité de direction d'une heure, enregistrée (avec l'accord des participants).

Outil. `transcrire-et-resumer.sh` + `prompts/compte-rendu.md`. Pour une heure d'audio, `resumer_long.py` découpe la transcription en morceaux, résume chacun, puis fusionne.

Ce que vous devez obtenir sur la transcription fictive `exemples/reunion-chantier.txt` : 3 décisions, 4 actions avec **responsable et échéance** (dont « Karim, commande des menuiseries, avant le 14 novembre »), 1 point en suspens (validation du coloris par le client).

Grille de contrôle. - ☐ Chaque action a un responsable **cité dans la réunion** (l'IA tend à attribuer des tâches au hasard si personne n'est nommé : elle doit écrire « non attribué »). - ☐ Les dates sont celles qui ont été prononcées.

9.6 Toute profession : répondre à un courriel délicat

Situation. Un client conteste une facture sur un ton agressif.

Outil. `prompts/email-reponse.md` avec `assistant-fr`. Vos notes : « rappeler le devis signé, proposer un appel cette semaine ».

```
ollama run assistant-fr "$(cat prompts/email-reponse.md)
$(cat exemples/email-client.txt)
MES NOTES : rappeler le devis signé du 2 septembre ; proposer un appel cette semaine."
```

Ce que vous devez obtenir sur `exemples/email-client.txt` : une réponse de 8 à 12 lignes, qui reconnaît le désagrément sans reconnaître de faute, rappelle les faits (devis signé du 2 septembre) et propose un appel.

Grille de contrôle. - ☐ Aucun engagement (remise, remboursement) que vous n'avez pas décidé. - ☐ Le ton correspond à votre façon d'écrire : sinon, ajoutez au prompt deux courriels que vous avez réellement envoyés comme modèles de style.

9.7 Un serveur pour l'équipe

Un **Mac mini** ou **Mac Studio** peut servir l'IA à 3 à 15 personnes sur le réseau local :

1. Installer Ollama, définir `OLLAMA_HOST=0.0.0.0:11434` **uniquement** si le réseau est maîtrisé, et restreindre l'accès par pare-feu (ou mieux, ne l'ouvrir qu'à Open WebUI sur la même machine).
2. Installer Open WebUI sur ce Mac : comptes individuels, rôles, bases de connaissances partagées.
3. Réglages d'énergie : empêcher la mise en veille, redémarrage automatique après coupure.
4. `OLLAMA_NUM_PARALLEL` pour servir plusieurs requêtes simultanées (chaque requête parallèle consomme de la mémoire de contexte).

Pour dimensionner, partez du chapitre 2 bis : la vitesse de génération mesurée est partagée entre les utilisateurs simultanés.

9.8 Mesurer le gain avant de généraliser

Chronométrez trois tâches répétitives pendant deux semaines, avec et sans IA, **relecture comprise**. Le modèle de tableau est dans `kit/mesure-gain.csv`. C'est la meilleure base pour décider, et pour convaincre l'équipe.

10. Dépannage et performances

10.1 Problèmes fréquents

Symptôme	Cause probable	Solution
`could not connect to ollama` / connexion refusée	Le service n'est pas lancé	Ouvrir l'app Ollama ou `ollama serve`
Réponses très lentes (1–3 tok/s), Mac qui rame	Modèle trop gros, swap	`ollama ps` ; prendre un modèle plus petit ou un MoE, fermer les apps lourdes
Le modèle « oublie » le début	Contexte trop court	Augmenter `num_ctx` (Modelfile) ou `OLLAMA_CONTEXT_LENGTH`
Mac qui chauffe, ventilateurs	Normal sous charge prolongée	Sur MacBook Air (sans ventilateur), la vitesse baisse après quelques minutes : préférer de plus petits modèles
Réponses en anglais	Prompt système absent	Ajouter « Réponds toujours en français » dans le SYSTEM (Modelfile)
Réponses inventées	Limite des modèles	Fournir les sources (RAG), demander « dis "je ne sais pas" si absent », baisser la température
`Address already in use`	Port déjà utilisé (8080…)	Changer le port : `--port 8081`
Disque plein	Modèles volumineux	`ollama list` puis `ollama rm` ; déplacer avec `OLLAMA_MODELS`
Whisper : texte vide ou absurde	Mauvais format audio	Reconvertir en WAV 16 kHz mono (chapitre 5.2)
Open WebUI : erreur à l'installation	Version de Python	Utiliser Python 3.11 ou 3.12 dans un venv

10.2 Diagnostiquer

```
ollama ps                      # modèle chargé, taille, moteur, % GPU
sysctl hw.memsize              # mémoire totale (octets)
memory_pressure                # pression mémoire
sudo powermetrics --samplers gpu_power -n 1   # charge GPU (admin)
```

Dans le **Moniteur d'activité**, l'onglet Mémoire doit rester vert/jaune ; si la « mémoire swap utilisée » grimpe de plusieurs Go pendant la génération, le modèle est trop gros.

Le script `kit/scripts/bench.sh` mesure la vitesse d'un modèle via les compteurs renvoyés par l'API (`eval_count`, `eval_duration`).

10.3 Gagner en vitesse

1. Préférer un **modèle MoE** (qwen3.6:35b, gemma4:26b) à un modèle dense de même qualité.
2. Rester en **Q4** (ou MLX 4-bit).
3. Ne pas surdimensionner le contexte.
4. Garder le modèle chargé (`OLLAMA_KEEP_ALIVE`).
5. Désactiver le mode « réflexion » pour les tâches simples.

6. Sur un Mac de bureau, laisser de la marge mémoire : fermer les navigateurs gourmands.
7. Avancé : `sudo sysctl iogpu.wired_limit_mb=<valeur>` permet d'augmenter la part de mémoire allouable au GPU (réinitialisée au redémarrage). À n'utiliser qu'en laissant au moins 8 Go au système.

10.4 Mettre à jour

- Ollama : l'application se met à jour seule (ou `brew upgrade ollama`) ; `ollama pull <modèle>` récupère la dernière version d'un modèle.
- LM Studio : mise à jour intégrée ; les moteurs (runtimes) se mettent à jour séparément dans les réglages.
- Python : `pip install -U mlx-lm mlx-whisper open-webui` dans les venvs correspondants.

10 bis. Arbre de dépannage et preuve de confidentialité

Partir du symptôme

A. « command not found: ollama » 1. L'application Ollama a-t-elle été lancée au moins une fois ? → Lancez-la depuis Applications et acceptez l'installation de la commande. 2. Toujours absent ? → Fermez et rouvrez le Terminal. Puis `ls /usr/local/bin/ollama`. 3. Installé avec Homebrew ? → `brew install ollama` puis `brew services start ollama`.

B. « could not connect to ollama app » / connexion refusée 1. L'icône du lama est-elle dans la barre des menus ? Sinon, lancez l'application. 2. `curl http://127.0.0.1:11434` doit répondre `Ollama is running`. 3. Sinon, un autre programme occupe le port : `lsof -i :11434`.

C. La réponse est très lente (moins de 5 tok/s) ou le Mac se fige 1. `ollama ps` : la colonne SIZE dépasse-t-elle environ 70 % de votre mémoire ? → Modèle trop gros : descendez d'une taille (chapitre 2 bis). 2. Moniteur d'activité › Mémoire : la pression est rouge, ou la mémoire swap augmente ? → Fermez navigateur et visioconférence, ou réduisez `num_ctx`. 3. La colonne PROCESSOR indique du CPU ? → Le modèle déborde du GPU : même remède que 1. 4. MacBook Air après plusieurs minutes ? → Ralentissement thermique normal (pas de ventilateur) : préférez un modèle plus petit pour les longues tâches.

D. La réponse est fausse ou inventée 1. La réponse se trouve-t-elle dans le texte fourni ? Non → c'est une hallucination : fournissez la source (chapitre 6) et ajoutez « Si l'information est absente, écris : non trouvé ». 2. Oui, mais mal reprise → baissez la température à 0–0,2 ; passez à un modèle plus grand ; découpez le document. 3. Calcul faux → ne faites pas calculer le modèle : faites vérifier par le tableur ou le script (cas 9.2).

E. Le modèle « oublie » le début du document 1. Document plus long que le contexte : `ollama show <modèle>` indique le contexte maximal ; le contexte réellement utilisé est fixé par `num_ctx` (Modelfile) ou `OLLAMA_CONTEXT_LENGTH`. 2. Augmenter le contexte consomme de la mémoire : pour une heure de réunion, utilisez plutôt `resumer_long.py` (découpage).

F. Réponses en anglais, ou mélangées → Utilisez `assistant-fr` (Modelfile du kit) ; écrivez le prompt en français ; pour les petits modèles, répétez « Réponds en français » à la fin du prompt.

G. Whisper : texte vide, répétitions en boucle, langue erronée 1. Forcez la langue : `-l fr`. 2. Reconvertissez en WAV 16 kHz mono (le script le fait). 3. Longs silences → boucles : découpez l'audio, ou retirez les silences avec ffmpeg.

H. Disque plein → `ollama list`, puis `ollama rm <modèle>` ; pour stocker les modèles sur un SSD externe, définissez `OLLAMA_MODELS`.

Prouver que rien ne sort de votre Mac

La confidentialité ne doit pas être crue sur parole, elle se vérifie. Trois niveaux :

1. **Le test mode avion** (tout le monde) : Wi-Fi coupé, câble débranché, posez une question. Une réponse prouve que le calcul est local.
2. **Les connexions ouvertes** (2 minutes) : pendant une génération, dans le Terminal :

```
lsof -i -P -n | grep -i -E "ollama|lm" | grep -v 127.0.0.1
```

Aucune ligne → aucune connexion en dehors de votre Mac pendant l'usage. (L'application peut contacter Internet **au démarrage** pour vérifier les mises à jour : c'est attendu, et cela ne contient pas vos textes.)

1. **Le pare-feu applicatif** (pour un cabinet) : un pare-feu sortant comme LuLu (gratuit, Objective-See) ou Little Snitch (payant) permet de bloquer tout accès Internet d'Ollama et de LM Studio en dehors des mises à jour que vous déclenchez vous-même. Conservez une capture de la règle dans votre registre RGPD (chapitre 8) : c'est une preuve de mesure technique.

> **Attention** — Ollama propose aussi des modèles **cloud** (nom se terminant par `-cloud`) qui s'exécutent sur des serveurs distants. N'utilisez jamais un nom de modèle se terminant par `-cloud` pour des données confidentielles. `ollama ps` et le nom du modèle suffisent à le vérifier.

11. Checklists

Installation (1 heure)

- ☐ Vérifier puce et mémoire : `kit/scripts/check-mac.sh`
- ☐ Activer FileVault
- ☐ Installer Ollama et lancer l'application
- ☐ `ollama pull` du modèle adapté à sa mémoire (chapitre 2)
- ☐ `ollama pull bge-m3` (embeddings)
- ☐ Installer LM Studio (optionnel)
- ☐ Copier le kit dans `~/ia-locale-kit` et lancer `scripts/installer.sh`
- ☐ Créer le modèle personnalisé : `ollama create assistant-fr -f modelfiles/assistant-fr.Modelfile`
- ☐ Tester le banc d'essai (`prompts/banc-essai-fr.md`)
- ☐ Test Wi-Fi coupé : tout fonctionne

Transcription

- ☐ ffmpeg installé
- ☐ whisper.cpp ou MLX Whisper + modèle large-v3-turbo
- ☐ Test sur un enregistrement de 5 minutes
- ☐ Mention d'information des participants préparée
- ☐ Règle de suppression des audios définie

RAG

- ☐ Documents triés, doublons supprimés, PDF scannés OCRisés
- ☐ Bases séparées par niveau de confidentialité
- ☐ Modèle d'embeddings multilingue (bge-m3)
- ☐ 10 questions de test avec réponses connues : taux de bonnes réponses noté

Conformité

- ☐ Finalités écrites
- ☐ Fiche registre complétée (`rgpd/fiche-registre.md`)
- ☐ Charte d'usage diffusée (`rgpd/charte-usage-ia.md`)
- ☐ Durées de conservation des historiques, transcriptions, index
- ☐ Sauvegardes chiffrées
- ☐ Personnes formées (maîtrise de l'IA, AI Act art. 4) — garder une trace

Avant chaque usage sensible

- ☐ Est-ce bien un modèle local (pas « cloud ») ?
- ☐ Ai-je minimisé les données (pseudonymisation) ?
- ☐ Vais-je relire et vérifier chaque fait, date, chiffre, citation ?
- ☐ La décision finale reste-t-elle humaine ?

Annexe A — Glossaire

Terme	Définition
API	Interface permettant à un programme d'en appeler un autre (ici : vos scripts appellent Ollama)
Contexte	Quantité de texte (en jetons) que le modèle prend en compte à la fois
Embedding	Représentation numérique du sens d'un texte, utilisée pour la recherche
GGUF	Format de fichier de modèles de llama.cpp, utilisé par Ollama et LM Studio
Hallucination	Réponse fausse formulée avec assurance
Jeton (token)	Unité de texte traitée par le modèle (≈ ¾ de mot en français)
LLM	Grand modèle de langage
Mémoire unifiée	Mémoire partagée entre processeur et GPU sur les puces Apple
MLX	Cadre de calcul d'Apple optimisé pour les puces Apple
MoE	« Mixture of Experts » : seule une partie du modèle travaille pour chaque jeton, d'où plus de vitesse
Modelfile	Fichier de configuration d'un modèle personnalisé dans Ollama
Paramètres (B)	Nombre de « poids » du modèle, en milliards
Quantisation	Compression des poids sur moins de bits (Q4, Q8…)
RAG	Recherche de passages dans vos documents puis génération d'une réponse sourcée
Température	Réglage de créativité : bas = factuel, haut = varié
tok/s	Jetons générés par seconde : la vitesse perçue

Annexe B — Contenu du kit

Fichier	Rôle	Chapitre
`scripts/check-mac.sh`	Diagnostic du Mac et modèle conseillé	2
`scripts/installer.sh`	Installation guidée (avec confirmations)	3
`scripts/ia.sh`	Consigne + texte → réponse (Raccourcis, Services)	7
`scripts/transcrire-et-resumer.sh` + `resumer_long.py`	Audio → transcription → compte rendu (prompt au choix)	5, 9
`scripts/bench.sh`	Mesure de vitesse	2 bis, 10
`scripts/extraire_facture.py`	Facture → ligne CSV contrôlée (JSON contraint)	9.2
`exemples/*.txt`	7 documents d'essai fictifs pour les 6 cas	9
`mesure-gain.csv`	Tableau de mesure du temps gagné	9.8
`modelfiles/assistant-fr.Modelfile`	Assistant généraliste francophone	3
`modelfiles/correcteur.Modelfile`	Correcteur orthographique	7
`modelfiles/documents.Modelfile`	Réponses strictement sourcées	6
`prompts/*.md`	8 prompts métier + banc d'essai	4, 9
`rag/rag_local.py`	RAG minimal local	6
`rgpd/*.md`	Checklist, fiche registre, charte	8

Le script ia.sh

```
#!/bin/bash
# usage : echo "texte" | ia.sh "Consigne" [modèle]
CONSIGNE="${1:-Réponds en français :}"; MODELE="${2:-${IA_MODELE:-assistant-fr}}"
TEXTE=$(cat)
python3 - "$CONSIGNE" "$MODELE" "$TEXTE" <<'PY'
import json, sys, urllib.request
c, m, t = sys.argv[1:4]
d = json.dumps({"model": m, "prompt": f"{c}\n\n{t}", "stream": False}).encode()
r = urllib.request.Request("http://127.0.0.1:11434/api/generate", d,
                           {"Content-Type": "application/json"})
print(json.load(urllib.request.urlopen(r, timeout=600))["response"].strip())
PY
```

Le cœur du RAG (extrait de rag_local.py)

```
def ask(question, k=5):
    data = json.loads(INDEX.read_text())
    q = embed([question])[0]                        # empreinte de la question
    top = sorted(data["items"], key=lambda it: cos(q, it["v"]), reverse=True)[:k]
    contexte = "\n\n".join(f"[{t['source']}]\n{t['texte']}" for t in top)
    prompt = ("Réponds en français uniquement à partir des extraits ci-dessous. "
              "Cite [fichier] après chaque affirmation. Si l'information n'y est pas, dis-
le.\n\n"
              f"EXTRAITS :\n{contexte}\n\nQUESTION : {question}")
    r = post("/api/chat", {"model": CHAT, "stream": False,
                           "messages": [{"role": "user", "content": prompt}]})
    print(r["message"]["content"].strip())
```

Prompt « compte rendu »

```
Tu reçois la transcription (ou des notes) d'une réunion professionnelle.
Rédige un compte rendu en français :
1. Synthèse (5 lignes maximum)
2. Décisions prises (liste)
3. Actions : tableau | Qui | Quoi | Échéance | (n'invente aucune échéance)
4. Points ouverts / questions
N'ajoute aucune information absente de la transcription.
```

Programme de prise en main sur 5 jours

Jour	Objectif	Durée
1	Installer Ollama + un modèle, test « Wi-Fi coupé »	45 min
2	Banc d'essai de 2–3 modèles, choisir le sien, créer `assistant-fr`	45 min
3	Raccourci « Reformuler » + « Corriger »	30 min
4	Transcrire une vraie réunion et produire le compte rendu	45 min
5	Indexer un dossier de procédures, 10 questions de test, remplir la checklist RGPD	1 h

Annexe C — Sources

Consultées le 30 octobre 2026 ; commandes, versions et modèles revérifiés le 31 octobre 2026.

- Ollama — bibliothèque de modèles et documentation : https://ollama.com/library, https://docs.ollama.com
- LM Studio : https://lmstudio.ai, https://lmstudio.ai/docs
- llama.cpp : https://github.com/ggml-org/llama.cpp
- MLX / mlx-lm : https://github.com/ml-explore/mlx-lm ; modèles : https://huggingface.co/mlx-community
- whisper.cpp : https://github.com/ggml-org/whisper.cpp ; MLX Whisper : https://pypi.org/project/mlx-whisper/
- Open WebUI : https://docs.openwebui.com
- LLMCheck, index des modèles locaux pour Mac (octobre 2026) : https://llmcheck.net
- Apple, fiches techniques Mac : https://support.apple.com/specs
- Mesures de vitesse Apple Silicon (chapitre 2 bis) : https://github.com/ggml-org/llama.cpp/discussions/4167
- Outils de pare-feu sortant : https://objective-see.org/products/lulu.html — https://www.obdev.at/products/littlesnitch
- RGPD (règlement (UE) 2016/679) et CNIL : https://www.cnil.fr
- Règlement (UE) 2024/1689 sur l'IA : https://eur-lex.europa.eu/eli/reg/2024/1689/oj
- Loi n° 71-1130 du 31 décembre 1971, Code de la santé publique : https://www.legifrance.gouv.fr

Les noms de produits cités appartiennent à leurs propriétaires respectifs. Ce guide est indépendant et n'est affilié à aucun éditeur. Les performances varient selon la machine, le modèle et les versions des logiciels.

Édition, mises à jour et contact

Cette édition

- **Version 2.0 — 31 octobre 2026.** Ajouts : démarrage express en 30 minutes, chapitre de mesures de vitesse sourcées, six cas concrets avec documents d'essai et grilles de contrôle, arbre de dépannage, preuve de confidentialité, script d'extraction de factures.
- Version 1.0 — 30 octobre 2026. Première édition.

Toutes les commandes et tous les noms de modèles de cette édition ont été vérifiés le 31 octobre 2026 sur les pages officielles (ollama.com/library, notes de version Ollama 0.40.13, lmstudio.ai, dépôts Hugging Face cités en annexe C).

Mises à jour

L'IA locale évolue chaque mois. Les acheteurs reçoivent gratuitement chaque nouvelle édition pendant 12 mois, avec un journal des changements. **[À VALIDER par l'auteur avant publication]**

Une erreur, une question ?

Écrivez à l'adresse indiquée sur votre reçu d'achat **[À VALIDER]** en indiquant le numéro de page. Les corrections signalées sont intégrées à l'édition suivante.